COMS W4167: Computer Animation

Programming Assignment 4

Due 9:00PM Tue. April 7th, 2026 (EST)

Academic Honesty Policy

You are permitted and encouraged to discuss your work with other students. Except where explicitly stated otherwise, you may work out equations in writing on paper or a whiteboard. You are encouraged to use Ed Discussion to converse with other students, the TAs, and the instructor.

HOWEVER, you must NOT share source code or hard copies of source code. Refrain from activities or the sharing materials that could cause your source code to APPEAR TO BE similar to another student's enrolled in this or previous years. We will be monitoring source code for individuality. Cheating will be dealt with severely. Source code should be yours and yours only. Do not cheat. For more details, please refer to the full academic honesty policy on the Engineering School's website.

Chapter 1

Course Notes

In this assignment, we will explore methods for simulating rigid bodies—objects in which the distance between any two points is fixed for all time. Rigid bodies have a number of interesting applications ranging from their use in robot motion planning to studying the time evolution of tops. Rigid bodies are also one of the most common primitives simulated in video game physics engines.

Rigid bodies see heavy use as they posses many favorable numeric properties; simulating similar systems with masses and springs or finite elements will lead to very stiff systems and hence strict time step limits. As a *reduced coordinate* representation, rigid bodies also have the potential to yield large savings due to the reduction in the number of degrees of freedom.

The use of *reduced* or *generalized* coordinates can be viewed as a constraint enforcement method. For example, one can encode the position of a pendulum with one variable that changes with time—the deflection of the pendulum arm from the vertical. The position of the pendulum bob can be recovered using simple trigonometry, the angle of deflection, and the length of the arm. By construction, the inextensibility of the arm cannot be violated. In contrast, if one were to use cartesian coordinates, the inextensibility of the constraint would have to be enforced in some manner. Usually reduced coordinates are difficult to write down for complex constrained systems, but for rigid bodies they prove both manageable and advantageous.

1.1 Rigid Body Dynamics

A rigid body is a collection of masses (or a continuum, but here we consider only a finite number of points) in which the distance between any two masses remains fixed for all time. Intuitively, the two transformations that preserve this invariant are translations and rotations. Therefore, not surprisingly, we will find that we can decompose the dynamics of a rigid body into two components: a component associated with the center of mass that behaves like a point mass, and a component associated with rotations about the center of mass.

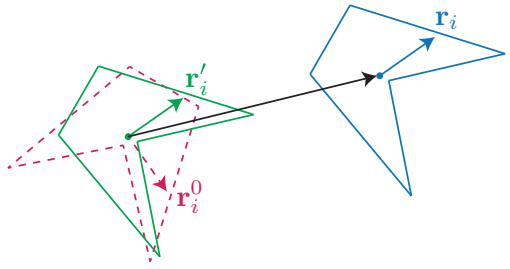

Figure 1.1: The *body space* and *world space* frames. $\boldsymbol{r}'_i = \mathbf{R}(t)\boldsymbol{r}^0_i$ and $\boldsymbol{r}_i = \mathbf{R}(t)\boldsymbol{r}^0_i + \boldsymbol{X}(t)$.

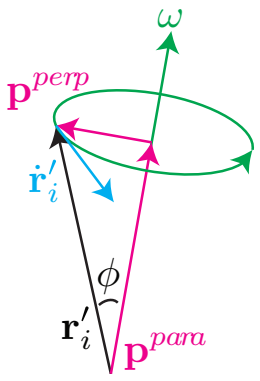

Figure 1.2: A rigid body rotating about its center of mass and axis ω.

1.1.1 Body Space, World Space, and Center of Mass

Consider a 3D rigid body composed of N points masses lying in the body. Recall that we define the center of mass of a collection of masses as

$$\boldsymbol{X} = \frac{\sum_i m_i \boldsymbol{r}_i}{\sum_i m_i} = \frac{\sum_i m_i \boldsymbol{r}_i}{M} \tag{1.1}$$

where m_i is the mass of the i^{th} particle, $\boldsymbol{r}_i$ is the position in *world space* of the i^{th} particle, and $M = \sum_i m_i$ is the total mass.

Define *body space* as a 3D orthonormal coordinate system with the origin placed at the body's center of mass and in which the body is considered to have no rotation (see Figure 1.1). If $\boldsymbol{r}_i^0$ is the position of any point on the rigid body in *body space*, then the point's position in *world space* is given by

$$\boldsymbol{r}_i = \mathbf{R}(t)\boldsymbol{r}_i^0 + \boldsymbol{X}(t) \tag{1.2}$$

where $\mathbf{R}(t)$ is a 3×3 rotation matrix describing the orientation of the rigid body at time t, and $\boldsymbol{X}(t)$ is the position of the body's center of mass at time t. As $\mathbf{R}$ is purely a rotation, we can represent it using a rotation-angle vector $\boldsymbol{\theta}$ (i.e., a 3D vector), where the unit vector $\boldsymbol{\theta}/|\boldsymbol{\theta}|$ is the axis of rotation and the magnitude $|\boldsymbol{\theta}|$ is the angle of rotation.

What does this construction gain us? At any time t, if we know the orientation of the body $\boldsymbol{\theta}$ and the center of mass' position $\boldsymbol{X}$, we can compute the position of any particle on the rigid body. Instead of simulating $3N$ degrees of freedom, we need only simulate 6 (3 for the center of mass' position and 3 for the orientation $\boldsymbol{\theta}$). Furthermore, as our representation only admits rigid motions, that is rotations and translations, we do not need to enforce the rigidity constraint with soft constraints (e.g. stiff forces), hard constraints (e.g. Lagrange multipliers), or some other technique.

Before we can simulate a rigid body with this *reduced coordinate* representation, however, we need to ascertain how the velocity of a particle on the body relates to this reduced representation, and how forces act on this reduced representation.

1.1.2 Linear and Angular Velocity

We can compute the velocity of a particle attached to a rigid body by taking the time derivative of (1.2):

$$\dot{\boldsymbol{r}}_i = \dot{\mathbf{R}}(t)\boldsymbol{r}_i^0 + \dot{\boldsymbol{X}}(t)$$

Note that $\boldsymbol{r}_i^0$ is fixed throughout the simulation and thus its time derivative is 0. $\dot{\boldsymbol{X}}$ is simply the velocity of the rigid body's center off mass, but how do we compute $\dot{\mathbf{R}}$? Before proceeding, let us examine the columns of $\mathbf{R}$.

Component-wise, we can write $\mathbf{R}$ as:

$$\begin{pmatrix} R_{xx} & R_{yx} & R_{zx} \\ R_{xy} & R_{yy} & R_{zy} \\ R_{xz} & R_{yz} & R_{zz} \end{pmatrix}$$

Consider the action of $\mathbf{R}$ on the cartesian x-axis:

$$\mathbf{R}\hat{\mathbf{x}} = \begin{pmatrix} R_{xx} & R_{yx} & R_{zx} \\ R_{xy} & R_{yy} & R_{zy} \\ R_{xz} & R_{yz} & R_{zz} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} R_{xx} \\ R_{xy} \\ R_{xz} \end{pmatrix}$$

That is, the first column of $\mathbf{R}$ is simply the *body space* x-axis rotated into world space! Similarly, we find that the second column of $\mathbf{R}$ is the *body space* y-axis rotated into world space. Therefore, if we can compute how these column vectors evolve instantaneously, we can compute $\dot{\mathbf{R}}$.

Now, consider a rigid body rotating about its center of mass and some axis $\boldsymbol{\omega}$ with angular velocity $\omega = |\boldsymbol{\omega}|$ (see Figure 1.2). Consider a vector pointing from the center of mass to some point on the body, say $\boldsymbol{r}_i' = \mathbf{R}\boldsymbol{r}_i^0 = (\boldsymbol{r}_i - \boldsymbol{X}) = \boldsymbol{p}^{perp} + \boldsymbol{p}^{para}$, where $\boldsymbol{p}^{perp}$ is perpendicular to $\boldsymbol{\omega}$ and $\boldsymbol{p}^{para}$ is parallel to $\boldsymbol{\omega}$. $\boldsymbol{p}^{para}$ is unaffected by the rotation, while $\boldsymbol{p}^{perp}$ will trace out a circle. Simple trigonometry gives that radius of the circle is $|\boldsymbol{r}_i - \boldsymbol{X}| \sin\phi$, where ϕ is the angle between $(\boldsymbol{r}_i - \boldsymbol{X})$ and $\boldsymbol{\omega}$. Thus, the instantaneous speed of any point on this circle is given by $|\boldsymbol{\omega}||\boldsymbol{r}_i - \boldsymbol{X}| \sin\phi$. Furthermore, the direction of the velocity of this point is perpendicular to both $\boldsymbol{\omega}$ and $(\boldsymbol{r}_i - \boldsymbol{X})$. What operation gives this magnitude and this direction? The cross product! Therefore, the velocity of $\boldsymbol{r}_i - \boldsymbol{X}$ is given by $\boldsymbol{\omega} \times (\boldsymbol{r}_i - \boldsymbol{X})$.

As a notational convenience, note that we can represent a cross product as a matrix-vector multiplication:

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix} = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix} \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} = \mathbf{a}^* \mathbf{b}$$

Returning to the problem of computing $\dot{\mathbf{R}}$, first note that translations of the rigid body do not change $\mathbf{R}$, and thus we need only consider changes to $\mathbf{R}$ due to the angular velocity. Invoking the above result:

$$\dot{\mathbf{R}} = \begin{pmatrix} \boldsymbol{\omega} \times \begin{pmatrix} R_{xx} \\ R_{xy} \\ R_{xz} \end{pmatrix} & \boldsymbol{\omega} \times \begin{pmatrix} R_{yx} \\ R_{yy} \\ R_{yz} \end{pmatrix} & \boldsymbol{\omega} \times \begin{pmatrix} R_{zx} \\ R_{zy} \\ R_{zz} \end{pmatrix} \end{pmatrix} = \boldsymbol{\omega}^* \mathbf{R}$$

Thus, the *world space* velocity of i^{th} particle in the rigid body is

$$\dot{\boldsymbol{r}}_i = \boldsymbol{\omega}(t)^* \mathbf{R}(t) \boldsymbol{r}_i^0 + \dot{\boldsymbol{X}}(t) = \boldsymbol{\omega}(t) \times \boldsymbol{r}_i'(t) + \dot{\boldsymbol{X}}(t).$$

where $\boldsymbol{r}_i' = R(t)\boldsymbol{r}_i^0$ denote the *body space* coordinate of particle i rotated into *world space* by the body's orientation. That is, if in addition to $\boldsymbol{X}(t)$ and $\boldsymbol{\theta}(t)$ we know $\dot{\boldsymbol{X}}(t)$ and $\omega(t)$, we can recover the velocity of any point on the rigid body! Therefore, we can represent the entire state of the rigid body with these four quantities.

If we wanted to simulate rigid bodies without any forces, the above would be enough; $\dot{\boldsymbol{X}}$ and $\boldsymbol{\omega}$ would remain constant throughout the simulation, and we would just update $\boldsymbol{X}(t)$ and $\boldsymbol{\theta}(t)$ using these constant values. This situation is not terribly interesting, however, so we will now see how Newton's second law ($\mathbf{F} = m\mathbf{a}$) looks when phrased in terms of our reduced coordinates.

1.1.3 Forces and Torques

Center of Mass Acceleration

To determine how a force effects the center of mass' velocity, $\dot{\boldsymbol{X}}$, we will start by computing the total momentum of the rigid body. The total momentum $\boldsymbol{P}$ of a rigid body is given by:

$$\boldsymbol{P} = \sum_i m_i \dot{\boldsymbol{r}}_i = \left(\sum_i m_i \right) \frac{\sum_i m_i \dot{\boldsymbol{r}}_i}{\sum_i m_i} = M\dot{\boldsymbol{X}} \tag{1.3}$$

That is, the momentum of the system can be computed by viewing all of the body's mass as moving with the center of mass. Taking the derivative of this quantity gives us the center of mass' acceleration:

$$M\ddot{\boldsymbol{X}} = \sum_i m_i \ddot{\boldsymbol{r}}_i = \sum_i \boldsymbol{f}_i$$

$\boldsymbol{f}_i$ is the total force acting on mass i. We have to be a little careful going forward, as this force includes both external forces (e.g. gravity) as well as the internal forces acting within the rigid body. Let us assume that each point j exerts a force on each other point i to maintain the rigidity constraint. That is, we can write the force on particle i as $\boldsymbol{f}_i = \boldsymbol{f}_i^{ext} + \sum_{j \neq i} \boldsymbol{f}_{ij}$ where $\boldsymbol{f}_i^{ext}$ is the sum of the external forces acting on i, and $\boldsymbol{f}_{ij}$ is the internal force on i from point j. Substituting this sum into the rate of change of momentum:

$$M\ddot{\boldsymbol{X}} = \sum_i \boldsymbol{f}_i = \sum_i (\boldsymbol{f}_i^{ext} + \sum_{j \neq i} \boldsymbol{f}_{ij}) = \sum_i \boldsymbol{f}_i^{ext} + \sum_i \sum_{j \neq i} \boldsymbol{f}_{ij}$$

Let us examine $\sum_i \sum_{j \neq i} \boldsymbol{f}_{ij}$ in more detail. We can rewrite this sum and invoke Newton's third law ('for every action there is an opposite and equal reaction'), giving:

$$\sum_i \sum_{j \neq i} \boldsymbol{f}_{ij} = \sum_i \sum_{j > i} (\boldsymbol{f}_{ij} + \boldsymbol{f}_{ji}) = \sum_i \sum_{j > i} (\boldsymbol{f}_{ij} - \boldsymbol{f}_{ij}) = 0$$

That is, provided the internal constraint forces obey Newton's third law, to compute the acceleration of the center of mass, we simply sum the external forces acting on each particle of the body:

$$M\ddot{\boldsymbol{X}} = \sum_i \boldsymbol{f}_i^{ext} = \mathbf{F}$$

Angular Acceleration

To determine the angular acceleration of a rigid body, we will first compute the time derivative of the angular momentum measured with respect to the origin. The total angular momentum of rigid body is given by

$$\boldsymbol{L} = \sum_i \boldsymbol{r}_i \times \boldsymbol{p}_i$$

and thus we compute the time derivative as:

$$\dot{\boldsymbol{L}} = \sum_i \dot{\boldsymbol{r}}_i \times \boldsymbol{p}_i + \sum_i \boldsymbol{r}_i \times \dot{\boldsymbol{p}}_i$$

In the first term, the velocity of a particle is parallel to its momentum, so this term is 0. In the second term, $\dot{\boldsymbol{p}}_i = \boldsymbol{f}_i = \boldsymbol{f}_i^{ext} + \sum_{j \neq i} \boldsymbol{f}_{ij}$ is the total force acting on the point, which as in the previous section includes both the external forces on the point and the internal constraint-maintaining forces. Making this substitution, we find:

$$\dot{\boldsymbol{L}} = \sum_i \boldsymbol{r}_i \times \dot{\boldsymbol{p}}_i = \sum_i \boldsymbol{r}_i \times (\boldsymbol{f}_i^{ext} + \sum_{j \neq i} \boldsymbol{f}_{ij}) = \sum_i \boldsymbol{r}_i \times \boldsymbol{f}_i^{ext} + \sum_i \boldsymbol{r}_i \times \sum_{j \neq i} \boldsymbol{f}_{ij}$$

Let us examine the second term in more detail. We can rearrange the sum and invoke Newton's third law to obtain:

$$\sum_i \boldsymbol{r}_i \times \sum_{j \neq i} \boldsymbol{f}_{ij} = \sum_i \sum_{j \neq i} \boldsymbol{r}_i \times \boldsymbol{f}_{ij} = \sum_i \sum_{j > i} (\boldsymbol{r}_i \times \boldsymbol{f}_{ij} + \boldsymbol{r}_j \times \boldsymbol{f}_{ji}) = \sum_i \sum_{j > i} ((\boldsymbol{r}_i - \boldsymbol{r}_j) \times \boldsymbol{f}_{ij})$$

When is this sum always 0? When $\boldsymbol{r}_i - \boldsymbol{r}_j$ is parallel to $\boldsymbol{f}_{ij}$. That is, when all constraint forces are central and obey Newton's third law, the time rate of change in angular momentum is simply the sum of all external torques:

$$\dot{\boldsymbol{L}} = \sum_i \boldsymbol{r}_i \times \boldsymbol{f}_i^{ext}$$

We can further decompose the angular momentum into a component associated with the center of mass' motion and a component associated with motion (spin) about the center of mass. Let $\boldsymbol{r}_i' = R(t)\boldsymbol{r}_i^0$ denote the *body space* coordinate of particle i rotated into *world space* by the body's orientation. Substituting a particle's position expressed relative to the center of mass into the formula for angular momentum:

$$\begin{aligned}
\boldsymbol{L} &= \sum_i \boldsymbol{r}_i \times \boldsymbol{p}_i = \sum_i (\boldsymbol{X} + \boldsymbol{r}_i') \times m_i(\dot{\boldsymbol{X}} + \dot{\boldsymbol{r}}_i') \\
&= \sum_i \boldsymbol{X} \times m_i\dot{\boldsymbol{X}} + \sum_i \boldsymbol{X} \times m_i\dot{\boldsymbol{r}}_i' + \sum_i \boldsymbol{r}_i' \times m_i\dot{\boldsymbol{X}} + \sum_i \boldsymbol{r}_i' \times m_i\dot{\boldsymbol{r}}_i' \\
&= \boldsymbol{X} \times (\sum_i m_i)\dot{\boldsymbol{X}} + \boldsymbol{X} \times (\sum_i m_i\dot{\boldsymbol{r}}_i') + (\sum_i m_i\boldsymbol{r}_i') \times \dot{\boldsymbol{X}} + \sum_i \boldsymbol{r}_i' \times m_i\dot{\boldsymbol{r}}_i' \\
&= \boldsymbol{X} \times \boldsymbol{P} + \boldsymbol{X} \times (\sum_i m_i\dot{\boldsymbol{r}}_i') + (\sum_i m_i\boldsymbol{r}_i') \times \dot{\boldsymbol{X}} + \sum_i \boldsymbol{r}_i' \times \boldsymbol{p}_i'
\end{aligned}$$

Let us look more closely at the two terms in parenthesis. Expanding $\sum_i m_i\boldsymbol{r}_i'$ gives:

$$\sum_i m_i\boldsymbol{r}_i' = \sum_i m_i(\boldsymbol{r}_i - \boldsymbol{X}) = \sum_i m_i\boldsymbol{r}_i - \sum_i m_i\boldsymbol{X} = (\sum_i m_i)\frac{\sum_i m_i\boldsymbol{r}_i}{\sum_i m_i} - (\sum_i m_i)\boldsymbol{X} = M\boldsymbol{X} - M\boldsymbol{X} = 0$$

$\sum_i m_i\dot{\boldsymbol{r}}_i'$ is simply the time derivative of this function, and hence is also 0. Thus, the total angular momentum of a rigid body with respect to the origin is given by:

$$\boldsymbol{L} = \boldsymbol{X} \times \boldsymbol{P} + \sum_i \boldsymbol{r}_i' \times \boldsymbol{p}_i' = \boldsymbol{L}^{point} + \boldsymbol{L}^{spin}, \tag{1.4}$$

where $\boldsymbol{P}$ is the linear momentum defined in (1.3).

Conveniently, the angular momentum decomposes into a component associated with the center of mass and the total momentum of the system, and a component associated with velocity relative to the center of mass. We will now compute $\dot{\boldsymbol{L}}^{point}$, and use this to obtain a simple formula for $\dot{\boldsymbol{L}}^{spin}$.

$$\dot{\boldsymbol{L}}^{point} = \dot{\boldsymbol{X}} \times \boldsymbol{P} + \boldsymbol{X} \times \dot{\boldsymbol{P}} = \boldsymbol{X} \times \mathbf{F} \tag{1.5}$$

Here we have used that $\dot{\boldsymbol{X}}$ is parallel to $\boldsymbol{P}$ and that $\dot{\boldsymbol{P}} = \mathbf{F}$ as proved in the previous section. We now compute $\dot{\boldsymbol{L}}^{spin}$ as

$$\dot{\boldsymbol{L}}^{spin} = \dot{\boldsymbol{L}} - \dot{\boldsymbol{L}}^{point} = \sum_i \boldsymbol{r}_i \times \boldsymbol{f}_i^{ext} - \boldsymbol{X} \times \mathbf{F} \tag{1.6}$$

$$= \sum_i (\boldsymbol{X} + \boldsymbol{r}_i') \times \boldsymbol{f}_i^{ext} - \boldsymbol{X} \times \mathbf{F} = \boldsymbol{X} \times \sum_i \boldsymbol{f}_i^{ext} + \sum_i \boldsymbol{r}_i' \times \boldsymbol{f}_i^{ext} - \boldsymbol{X} \times \mathbf{F} \tag{1.7}$$

$$= \boldsymbol{X} \times \mathbf{F} + \sum_i \boldsymbol{r}_i' \times \boldsymbol{f}_i^{ext} - \boldsymbol{X} \times \mathbf{F} = \sum_i \boldsymbol{r}_i' \times \boldsymbol{f}_i^{ext} \tag{1.8}$$

This is a remarkable result! If we compute the angular momentum in the non-inertial (accelerating) reference frame of the center of mass, the rate of change of the angular momentum takes the simple form $\dot{\boldsymbol{L}}^{spin} = \sum_i \boldsymbol{r}_i' \times \boldsymbol{f}_i^{ext}$, and the right hand side is the definition of the torque $\boldsymbol{\tau}$ acting on the body:

$$\boldsymbol{\tau} = \sum_i \boldsymbol{r}_i' \times \boldsymbol{f}_i^{ext}. \tag{1.9}$$

Moment of Inertia

The moment of inertia I of a rigid body is the "mass" of the body's rotation. Its definition starts from the rotational kinetic energy of the body. The rotational kinetic energy T of a rigid body is given by:

$$T = \frac{1}{2}\sum_i m_i\dot{\boldsymbol{r}}_i' \cdot \dot{\boldsymbol{r}}_i' = \frac{1}{2}\sum_i m_i(\boldsymbol{\omega} \times \boldsymbol{r}_i') \cdot (\boldsymbol{\omega} \times \boldsymbol{r}_i') = \frac{1}{2}\sum_i m_i(\hat{\boldsymbol{r}}_i'\boldsymbol{\omega}) \cdot (\hat{\boldsymbol{r}}_i'\boldsymbol{\omega}) = \boldsymbol{\omega}^T\left(\frac{1}{2}\sum_i m_i\hat{\boldsymbol{r}}_i'^T\hat{\boldsymbol{r}}_i'\right)\boldsymbol{\omega}, \tag{1.10}$$

where $\hat{\boldsymbol{r}}_i'$ is the cross-product matrix of $\boldsymbol{r}_i'$:

$$\hat{\boldsymbol{r}}_i' = \begin{pmatrix} 0 & -\boldsymbol{r}_{iz}' & \boldsymbol{r}_{iy}' \\ \boldsymbol{r}_{iz}' & 0 & -\boldsymbol{r}_{ix}' \\ -\boldsymbol{r}_{iy}' & \boldsymbol{r}_{ix}' & 0 \end{pmatrix} \tag{1.11}$$

Therefore, in 3D, the moment of inertia of a rigid body is a 3×3 matrix, defined as $\mathbf{I}_{\text{body}} = \sum_i -m_i \hat{\boldsymbol{r}}_i' \hat{\boldsymbol{r}}_i'$. Since the moment of inertia depends on $\boldsymbol{r}_i'$, it is a function of the body's orientation, and thus not a constant. If the rigid body is rotated by a rotation matrix R, the moment of inertia under that rotation is given by:

$$\mathbf{I}_{\text{body}} = \mathbf{R}\mathbf{I}_{\text{body}}\mathbf{R}^T. \tag{1.12}$$

For primitive shapes, such as a sphere, cylinder, or box, the moment of inertia can be computed analytically. For a sphere of radius R and mass M, the moment of inertia is given by:

$$\mathbf{I}_{\text{sphere}} = \frac{2}{5} M R^2 \mathbf{Id}. \tag{1.13}$$

1.1.4 Equations of Motion

To summarize, the equations of motion for our reduced coordinate representation of a rigid body are given by:

$$\frac{d}{dt} \begin{pmatrix} \boldsymbol{X} \\ \mathbf{R} \\ \boldsymbol{P} \\ \boldsymbol{L} \end{pmatrix} = \begin{pmatrix} \dot{\boldsymbol{X}} \\ \boldsymbol{\omega}^* \mathbf{R} \\ \mathbf{F} \\ \boldsymbol{\tau} \end{pmatrix} \tag{1.14}$$

In practice, we don't represent the rotational DoF using the rotational matrix $\mathbf{R}$, because a simple time-stepping scheme that updates $\mathbf{R}$ directly with a delta change may not preserve the orthonormality of the matrix, and a direct time derivatve of angular momentum $\boldsymbol{L} = \mathbf{I}\boldsymbol{\omega}$ is not easy to compute, as both $\mathbf{I}$ and $\boldsymbol{\omega}$ are functions of time.

1.1.5 Time Integration of the Equation of Motion

Quaternion. In practice and the programming assignment, we represent the rigid body rotation using a quaternion $\boldsymbol{q} = (q_w, q_x, q_y, q_z)$ (instead of the rotation matrix $\mathbf{R}$). The quaternion is a unit quaternion, representing a rotation around an axis $\mathbf{u}$ by an angle θ. The quaternion is given by:

$$\boldsymbol{q} = \left(\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\mathbf{u} \right). \tag{1.15}$$

If you are not familar with the concept of quaternion, please refer to this Wikipedia page. In the assignment code, the rigid body configuration is stored in a vector `state.shape_q`, which is a $N \times 7$ `numpy` array, where N is the number of rigid bodies. The first 3 elements in each row are the position of the body's center of mass, and the last 4 elements are the quaternion representing the body's rotation (a.k.a. the orientation).

But the velocity of a rigid body includes the translation velocity $\dot{\boldsymbol{X}}$ and the rotational velocity $\boldsymbol{\omega}$. Both are 3D vectors. In the assignment code, the rigid body velocity is stored in a $N \times 6$ `numpy` array, where the first 3 elements in each row are the translation velocity $\dot{\boldsymbol{X}}$, and the last 3 elements are the rotational velocity $\boldsymbol{\omega}$.

Time Integration. The time integration of the equation of motion needs to integrate both the translation and rotation of the rigid body. The update of translation component is straightforward. For example, in *Symplectic Euler*, the update of translation component at time step n is given by:

$$\dot{\boldsymbol{X}}_{n+1} = \dot{\boldsymbol{X}}_n + \frac{\mathbf{F}_n}{M} dt, \text{ and then } \boldsymbol{X}_{n+1} = \boldsymbol{X}_n + \dot{\boldsymbol{X}}_{n+1} dt. \tag{1.16}$$

where $\mathbf{F}_n$ is the total force acting on the body at time step n, and M is the mass of the body.

The update of rotation component is more complex, because the moment of inertia $\mathbf{I}$ is a function of the body's orientation. There are two ways to handle this. **Approach 1)** Update the moment of inertia first using Eq. (1.12), and then compute the angular acceleration $\dot{\boldsymbol{\omega}}$ using the formula $\dot{\boldsymbol{\omega}} = \mathbf{I}^{-1}\boldsymbol{\tau}$. But this approach is not efficient or stable, because it requires the update of the moment of inertia matrices at each time step. **We therefore take the second approach, which you are asked to implement in this programming assignment**.

Approach 2) We update the angular velocity in the body's frame of reference, and then convert it to the world frame of reference at the end. In this approach, because we update the angular velocity in the body's frame of reference, the moment of inertia matrix is constant! But the frame of reference is changing, so we need to consider the change of the frame of reference, and this brings an additional term in the force computation, known as the *Coriolis force*.

Consider the angular momentum $\boldsymbol{L} = \mathbf{I}\boldsymbol{\omega}$, which can be written in the body's local frame of reference too:

$$\boldsymbol{L} = \boldsymbol{L}_x\hat{\mathbf{i}} + \boldsymbol{L}_y\hat{\mathbf{j}} + \boldsymbol{L}_z\hat{\mathbf{k}}, \tag{1.17}$$

where $\hat{\mathbf{i}}, \hat{\mathbf{j}}, \hat{\mathbf{k}}$ are the axis vectors of the body's local frame. To find the time derivative of $\boldsymbol{L}$ as seen from the World Frame, we apply the product rule to the expression above. Note that in the World Frame, both the components and the basis vectors are changing over time.

$$\left(\frac{d\boldsymbol{L}}{dt}\right)_{\text{world}} = \underbrace{\left(\frac{d\boldsymbol{L}_x}{dt}\hat{\mathbf{i}} + \frac{d\boldsymbol{L}_y}{dt}\hat{\mathbf{j}} + \frac{d\boldsymbol{L}_z}{dt}\hat{\mathbf{k}}\right)}_{\text{Term 1}} + \underbrace{\left(\boldsymbol{L}_x\frac{d\hat{\mathbf{i}}}{dt} + \boldsymbol{L}_y\frac{d\hat{\mathbf{j}}}{dt} + \boldsymbol{L}_z\frac{d\hat{\mathbf{k}}}{dt}\right)}_{\text{Term 2}} \tag{1.18}$$

Term 1 represents the rate of change of $\boldsymbol{L}$'s coordinates relative to the rotating axes. This is exactly what an observer sitting on the rigid body would measure:

$$\text{Term 1} = \left(\frac{d\boldsymbol{L}}{dt}\right)_{\text{body}} = \begin{bmatrix}\hat{\mathbf{i}} & \hat{\mathbf{j}} & \hat{\mathbf{k}}\end{bmatrix}\mathbf{I}_{\text{body}}\dot{\boldsymbol{\omega}}_{\text{body}}, \tag{1.19}$$

where in the body's local frame, the moment of inertia matrix is constant, and the angular acceleration $\dot{\boldsymbol{\omega}}_{\text{body}}$ is expressed in the body's local frame.

Term 2 represents the change due to the rotation of the basis vectors themselves. For any rotating unit vector $\hat{\mathbf{u}}$ with angular velocity $\boldsymbol{\omega}$, the velocity of its tip is given by the cross product:

$$\frac{d\hat{\mathbf{u}}}{dt} = \boldsymbol{\omega} \times \hat{\mathbf{u}}$$

Substitute this into Term 2:

$$\text{Term 2} = \boldsymbol{L}_x(\boldsymbol{\omega} \times \hat{\mathbf{i}}) + \boldsymbol{L}_y(\boldsymbol{\omega} \times \hat{\mathbf{j}}) + \boldsymbol{L}_z(\boldsymbol{\omega} \times \hat{\mathbf{k}}) \tag{1.20}$$

Since the cross product is distributive, we can factor out $\boldsymbol{\omega}$:

$$\text{Term 2} = \boldsymbol{\omega} \times (\boldsymbol{L}_x\hat{\mathbf{i}} + \boldsymbol{L}_y\hat{\mathbf{j}} + \boldsymbol{L}_z\hat{\mathbf{k}}) = \boldsymbol{\omega} \times \boldsymbol{L} = \begin{bmatrix}\hat{\mathbf{i}} & \hat{\mathbf{j}} & \hat{\mathbf{k}}\end{bmatrix}(\boldsymbol{\omega}_{\text{body}} \times (\mathbf{I}_{\text{body}}\boldsymbol{\omega}_{\text{body}}))$$

Putting these two terms back into (1.18), we get:

$$\left(\frac{d\boldsymbol{L}}{dt}\right)_{\text{world}} = \begin{bmatrix}\hat{\mathbf{i}} & \hat{\mathbf{j}} & \hat{\mathbf{k}}\end{bmatrix}\boldsymbol{\tau}_{\text{body}} = \begin{bmatrix}\hat{\mathbf{i}} & \hat{\mathbf{j}} & \hat{\mathbf{k}}\end{bmatrix}(\mathbf{I}_{\text{body}}\dot{\boldsymbol{\omega}}_{\text{body}} + \boldsymbol{\omega}_{\text{body}} \times (\mathbf{I}_{\text{body}}\boldsymbol{\omega}_{\text{body}})\,.) \tag{1.21}$$

Writting it in the body's local frame, we have

$$\mathbf{I}_{\text{body}}\dot{\boldsymbol{\omega}}_{\text{body}} = \boldsymbol{\tau}_{\text{body}} - \boldsymbol{\omega}_{\text{body}} \times (\mathbf{I}_{\text{body}}\boldsymbol{\omega}_{\text{body}})\,. \tag{1.22}$$

Here the second term is the *Coriolis force* term. In summary, the time integration of the angular component of a rigid body has the following steps (here I use superscript n to denote the time step):

1. Convert torque from the world frame to the body's local frame $\boldsymbol{\tau}^n_{\text{body}} = \mathbf{R}^T \boldsymbol{\tau}^n_{\text{world}}$.

2. Convert the current angular velocity from the world frame to the body's local frame $\boldsymbol{\omega}^n_{\text{body}} = \mathbf{R}^T \boldsymbol{\omega}^n_{\text{world}}$.

3. cmpute the current angular acceleration in the body's local frame $\dot{\boldsymbol{\omega}}^n_{\text{body}} = \mathbf{I}^{-1}_{\text{body}} \left(\boldsymbol{\tau}^n_{\text{body}} - \boldsymbol{\omega}_{\text{body}} \times (\mathbf{I}_{\text{body}} \boldsymbol{\omega}_{\text{body}}) \right)$.

4. update the angular velocity in the body's local frame $\boldsymbol{\omega}^{n+1}_{\text{body}} = \boldsymbol{\omega}^n_{\text{body}} + \dot{\boldsymbol{\omega}}^n_{\text{body}} dt$.

5. convert the angular velocity back to the world frame $\boldsymbol{\omega}^{n+1}_{\text{world}} = \mathbf{R} \boldsymbol{\omega}^{n+1}_{\text{body}}$.

6. update the orientation quaternion $\boldsymbol{q}^{n+1} = \boldsymbol{q}(\boldsymbol{\omega}^{n+1}_{\text{world}} \cdot dt) \otimes \boldsymbol{q}^n$.

where $\boldsymbol{q}(\boldsymbol{\omega}^{n+1}_{\text{world}} \cdot dt)$ is the quaternion representing the rotation around the axis $\boldsymbol{\omega}^{n+1}_{\text{world}}$ by an angle $|\boldsymbol{\omega}^{n+1}_{\text{world}}| dt$—In practice, we use scipy.spatial.transform.Rotation.from_rotvec to convert $\boldsymbol{\omega}^{n+1}_{\text{world}} \cdot dt$ to a quaternion (see comments in the code). $\otimes$ is the quaternion multiplication operator, which represents the composition of two rotations.

An implemetation detail. In practice, we may artificially damp the angular velocity to prevent it from blowing up. Not only doing this will help the stability of time integrating the angular component, but also it will make the simulation more realistic, as in reality, forces such as air drags slow down the rigid body's rotation. A simple trick is to multiply the angular velocity by a damping factor $0 < \alpha < 1$ after the update, i.e. $\boldsymbol{\omega}^{n+1}_{\text{world}} = \alpha \boldsymbol{\omega}^{n+1}_{\text{world}}$ after Step 5 above. This is also used in many industrial rigid body simulators (such as Nvidia Newton). In the assignment code, I recommend to use the damping factor $\alpha = 0.999$.

1.2 Contacts Resolution

The next step of this assignement is to implement the contacts resolution for rigid bodies. I have already implemented instantaneous contacts detection for rigid bodies (not a super efficient one, but it is enough for the assignment). Similar to the last assignment, CollisionDetector.instantaneous_contacts(...) returns a Contacts object, which contains a list of detected contacts.

1.2.1 Penalty-based Contact Resolution

A simple way to resolve contacts is to use the penalty method, similar to what you have implemented in the last assignment. For each contact, you compute the penalty force $\mathbf{F}_{\text{penalty}}$ using the provided function penalty_force(...) in nemo/solvers/contact_penalty_solver.py. Then, you add the penalty force and the resulting torque $\boldsymbol{r}_i \times \mathbf{F}_{\text{penalty}}$ to the corresponding row of the state.shape_f array. This is very much similar to what you have implemented in the last assignment, and in this assignment, I provide you the solution of the last assignment in src/nemo/solvers/contact_penalty_solver.py.

1.2.2 Impulse-based Linear Complementarity Solver

A more efficient way to resolve contacts is to use the impulse-based linear complementarity solver. The solver you'll implement is a simplified version of the LCP solver that I discussed in the lecture—we don't consider friction in this assignment (in other words, the entire scene is frictionless).

The Linear Complementarity Problem. Consider a single contact i between two rigid bodies. We need to compute the contact impulse f_i and relative *normal* velocity v_i at the contact point. Here both f_i and v_i are scalars as we consider the normal direction only (i.e., $\boldsymbol{f}_i = f_i \boldsymbol{n}_i$, where $\boldsymbol{n}_i$ is the normal vector at the contact point). In this case, f_i and v_i satisfy a linear constraint:

$$A_i f_i + b_i = v_i, \tag{1.23}$$

where the coefficients A_i and b_i will be derived shortly. This is to say "if there is a contact impulse $\boldsymbol{f}_i$ in the normal direction, then the relative velocity at that contact point after $\boldsymbol{f}_i$ is applied is v_i".

In addition to the above linear constraint, we have the following complementarity condition:

$$f_i \geq 0, \quad v_i \geq 0, \quad f_i v_i = 0. \tag{1.24}$$

This is to say "if there is no contact impulse $\boldsymbol{f}_i$ in the normal direction, then the relative velocity must be already positive (i.e., receding contact)" or "if there is a contact impulse $\boldsymbol{f}_i$ in the normal direction, then the contact impulse is suppose to kill all relative velocity (not causing further penetration)". This corresponds to the maximal dissipation of energy at the contact point as we discussed in the lecture. Technically, $v_i \geq c$ for some positive c should be considered if we want to take into account the restitution effect. But for simplicity, we will not consider it in this assignment.

This LCP problem can be solved using a simple iterative solver, known as the Gauss-Seidel method. It taske maxits iterations, in each iteration:

1. iterate over all contacts. For each contact i:
   (a) If the relative velocity v_i is positive, then the contact impulse f_i is zero and move on to the next contact.
   (b) If the relative velocity v_i is negative, then compute contact impulse f_i, and use the f_i to update the linear and angular velocities of the two bodies at the contact point. If one of the bodies is a static object, then only update the linear and angular velocities of the other body.

The algorithm is to iteratively solve the LCP problem at each contact point. The LCP problem only update the velocities (linear and angular). Only after the LCP solver is done, we update the positions and rotations of the bodies using the updated velocities. This is a simplified version of the LCP solver that I discussed in the lecture, beacuse we don't consider friction and restitution, thereby avoiding the complexity introduced by the friction cone.

Now the question is how to compute the coefficients A_i and b_i in the linear constraint and perform Step **1.b** above. Consider a contact point i, let $\boldsymbol{r}$ be the vector from the center of mass of the first body to the contact point. Applying the impulse $\boldsymbol{f}_i$ at the contact point has two effects: It changes the linear velocity and the angular velocity of the first body.

$$\boldsymbol{f}_i = m_1 \Delta \boldsymbol{v}_1 \text{ and } \boldsymbol{r} \times \boldsymbol{f}_i = \mathbf{I}_1 \Delta \boldsymbol{\omega}_1, \tag{1.25}$$

where m_1 and $\mathbf{I}_1$ are the mass and moment of inertia of the first body, respectively ($\mathbf{I}_1$ is in the world frame under the current orientation). The velocity of a particle on the first body at the contact point is given by

$$\boldsymbol{v}_1 = \boldsymbol{v}_1^0 + \boldsymbol{\omega}_1 \times \boldsymbol{r}, \tag{1.26}$$

where $\boldsymbol{v}_1^0$ is the linear velocity of the center of mass of the first body and $\boldsymbol{\omega}_1$ is the angular velocity of the first body. Given the contact normal $\boldsymbol{n}_i$, the normal velocity change at the contact point i can be written as

$$\begin{aligned}
\Delta v_1 &= (\Delta \boldsymbol{v}_1 + \Delta \boldsymbol{\omega}_1 \times \boldsymbol{r}) \cdot \boldsymbol{n}_i \\
&= \left[\boldsymbol{f}_i m^{-1} + \left(\mathbf{I}_1^{-1}(\boldsymbol{r} \times \boldsymbol{f}_i)\right) \times \boldsymbol{r}\right] \cdot \boldsymbol{n}_i \\
&= f_i \left[\mathbf{n}_i m^{-1} + \left(\mathbf{I}_1^{-1}(\boldsymbol{r} \times \mathbf{n}_i)\right) \times \boldsymbol{r}\right] \cdot \boldsymbol{n}_i \\
&= f_i \left[\mathbf{n}_i \cdot \mathbf{n}_i m^{-1} + \left(\mathbf{I}_1^{-1}(\boldsymbol{r} \times \mathbf{n}_i)\right) \times \boldsymbol{r} \cdot \mathbf{n}_i\right] \\
&= f_i \left[m^{-1} + (\boldsymbol{r} \times \mathbf{n}_i)^T \mathbf{I}_1^{-1}(\boldsymbol{r} \times \mathbf{n}_i)\right] \\
&= f_i w_1
\end{aligned} \tag{1.27}$$

Here w_1 is the "mass-inverse" of the first body in the contact space, which is a scalar (i.e., $w_1 = m^{-1} + (\boldsymbol{r} \times \mathbf{n}_i)^T \mathbf{I}_1^{-1}(\boldsymbol{r} \times \mathbf{n}_i)$).

Similarly, you can compute w_2 for the second body in the contact space at this contact point i. Then the total normal velocity change at the contact point i is

$$\Delta v_i = f_i(w_1 + w_2). \tag{1.28}$$

Notice that if one of the bodies is a static object, say body 2, then the mass inverse (and inertia tensor inverse) is zero—you can think of a static object as having infinite mass, and thus $w_2 = 0$.

With w_1 and w_2 computed at each contact point, we can compute the contact impulse f_i needed to kill all relative velocity at the contact point i. At each iteration of the LCP solver, when considering the contact point i, we know the relative velocity v_i at the contact point i:

$$v_i = (\boldsymbol{v}_1 - \boldsymbol{v}_2) \cdot \mathbf{n}_i. \tag{1.29}$$

If $v_i < 0$, then the contact impulse f_i is needed to kill all relative velocity, and we have $f_i = -v_i/(w_1 + w_2)$, which is the f_i needed in the Step **1.b** above.

An implementation details. If you implement the LCP solver described above correctly, you'll likely experiencing the classic "sinking" problem in discrete physics simulation. Since you solve for velocity at the current frame to be zero, gravity immediately accelerates the object during the next time step, pushing it into the ground before the solver gets another chance to look at it.

The most common fix (used in Box2D, Bullet, etc.) is to bias the velocity to prevent penetration, known as *Baumgarte stabilization*. Instead of solving for a target relative velocity of 0, you solve for a small "rebound" velocity that is proportional to how deep the penetration already is. Instead of using $f_i = -v_i/(w_1 + w_2)$, you compute f_i using

$$f_i = \frac{-v_i + b}{w_1 + w_2}, \text{ where } b = \frac{\beta}{\Delta t}\text{penetration\_depth}. \tag{1.30}$$

where β is a scaling factor (usually between 0.1 and 0.8). It represents how much of the error you want to fix in a single frame. penetration_depth is the penetration depth at the contact point i, which is indicated by the contact_depth field in the Contacts object. In my implementation, I use $\beta = 0.5$.

Chapter 2

Assignment Requirements

In this assignment, we will simulate complex rigid objects beyond spheres. We will even load some triangle meshes for the rigid bodies using the trimesh library. The scene configuration files are self-explanatory. Please refer to files in scenes/pa4. This assignment has the following *three* parts.

2.1 Symplectic Time Integrator

Implement the symplectic time integrator in nemo/integrators/symplectic.py for rigid body dynamics. In particular, complete the integrate method according to the description in the lecture and in Section 1.1.5.

2.2 Penalty-based Collision Response

Compute the penalty force between rigid bodies and between rigid bodies and static objects, and use it to complete the step method in nemo/solvers/contact_penalty_solver.py. I provided the solution of this method in the last assignment for penalty-based collision response for particles. You can use it as a reference to implement the penalty force for rigid bodies. The major difference is that the penalty force for a rigid body also introduces a torque.

2.3 LCP Solver

Implement the LCP solver in nemo/solvers/lcp_gs.py according to the description in the lecture and in Section 1.2.2.